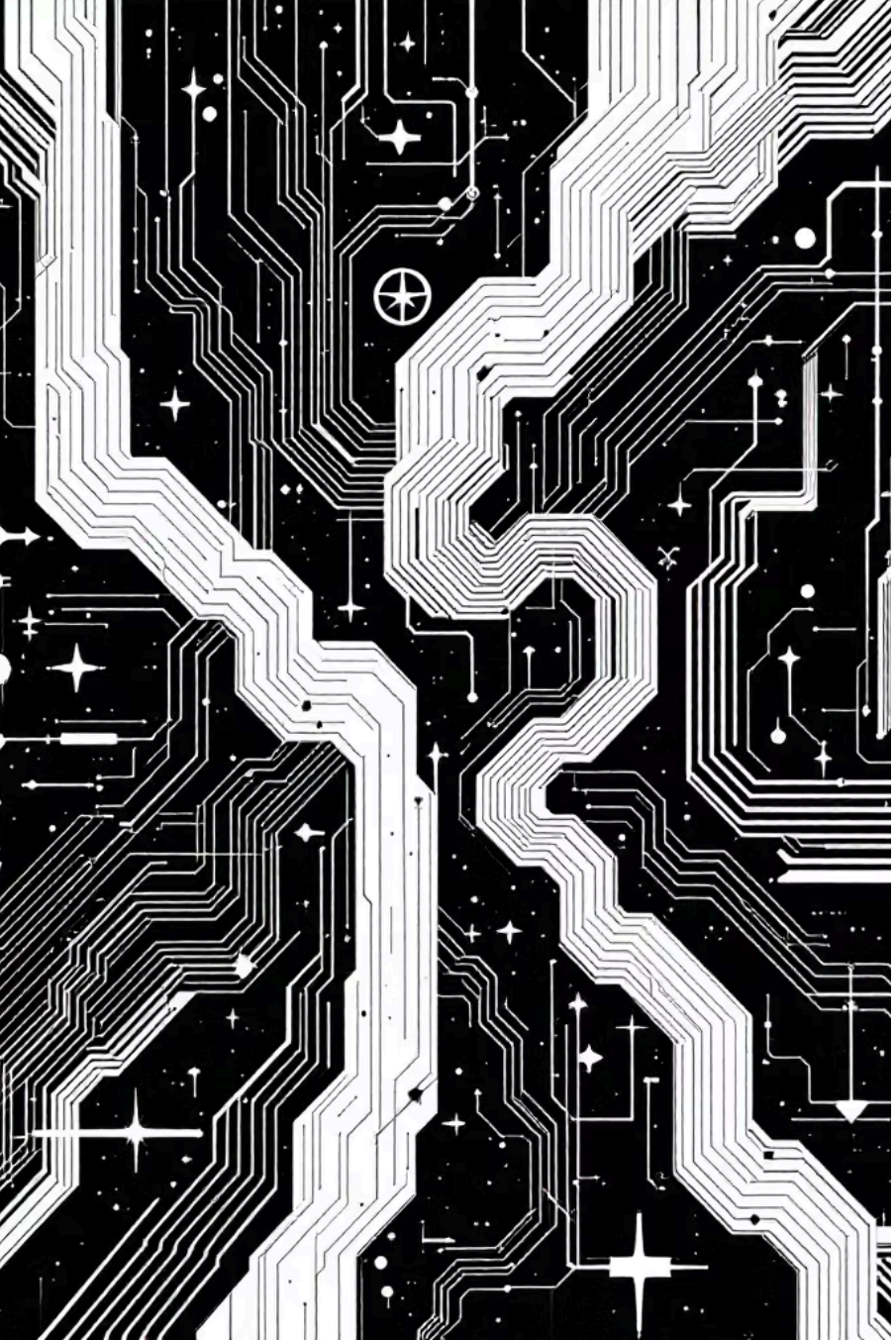


Introduction to Concurrency

Concurrency involves the simultaneous execution of multiple instruction sequences, typically seen in operating systems with parallel process threads. This fundamental concept allows for efficient utilization of computing resources and responsiveness in applications.

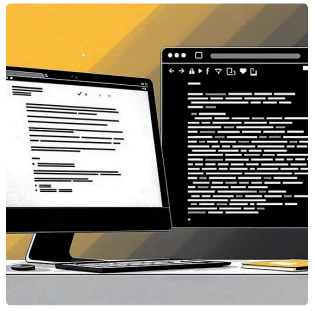
Made with GAMMA

Understanding Threads



Single Sequence Stream

A thread is a single sequence stream within a process, representing an independent path of execution. It's often referred to as a light-weight process.



Achieving Parallelism

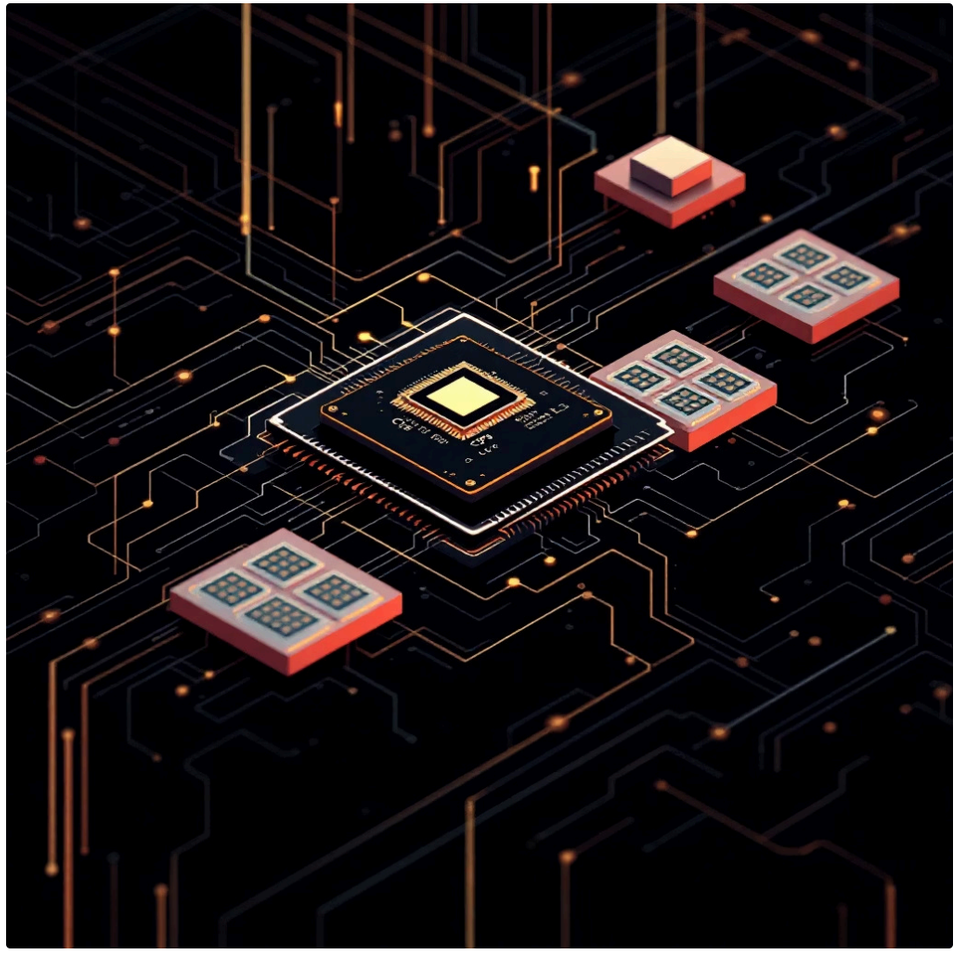
Threads achieve parallelism by dividing a process's tasks into independent execution paths. Examples include multiple browser tabs or concurrent spell-checking and formatting in a text editor.

Made with GAMMA

Thread Scheduling & Context Switching

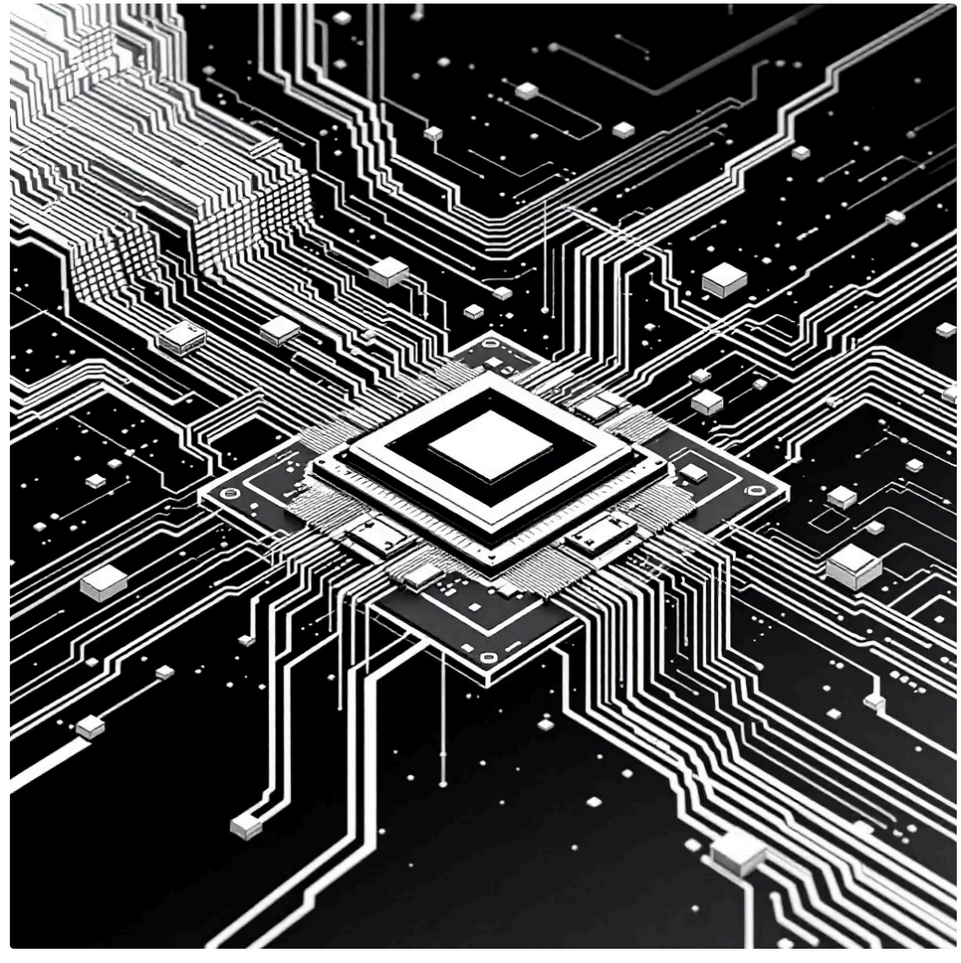
Thread Scheduling

Threads are scheduled for execution based on their priority. Even within a runtime, the operating system assigns processor time slices to all threads, managing their access to the CPU.



Context Switching

The OS saves the current thread's state and switches to another thread within the same process. This is a fast operation as it doesn't involve switching memory address space, though program counter, registers, and stack are included. CPU's cache state is preserved.



Made with GAMMA

CPU Access and State Management

1

Program Counter

Each thread possesses its own program counter, which dictates the next instruction to be executed.

2

OS Scheduling

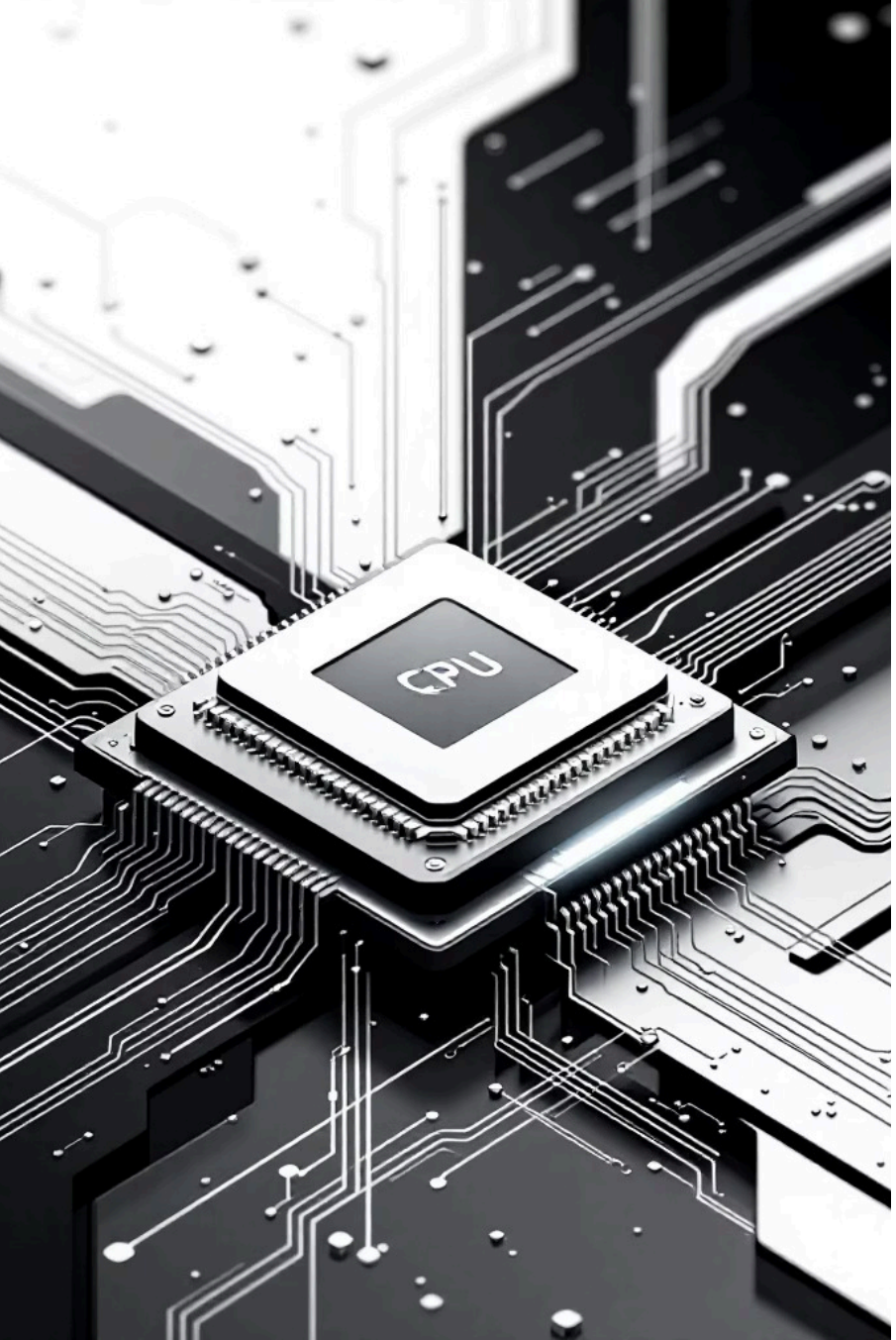
The operating system schedules these threads based on specific thread scheduling algorithms, fetching and executing instructions corresponding to each thread's program counter.

3

Context Switching

Context switching, often triggered by I/O or time quantum (TQ) expiration, uses a Thread Control Block (TCB) for state storage management, similar to a Process Control Block (PCB).

Made with GAMMA



Multi-threading on Single CPU Systems

A single CPU system does not gain performance from multi-threading. Since two threads must context switch for that single CPU, there is no actual gain in execution speed. Multi-threading's benefits are primarily realized in multi-processor or multi-core environments.

Made with GAMMA

Benefits of Multi-threading



Responsiveness

Multi-threading enhances application responsiveness by allowing parts of a program to continue execution even if other parts are blocked or performing lengthy operations.



Resource Sharing

Threads within the same process efficiently share resources like memory and open files, reducing overhead compared to creating separate processes.



Economy

Creating and context switching threads is more economical than processes. Allocating memory and resources for new processes is costly, making threads a better choice for task division within a process.



Multiprocessor Utilization

Threads allow for greater and more efficient utilization of multiprocessor architectures, truly leveraging parallel processing capabilities.

Made with GAMMA

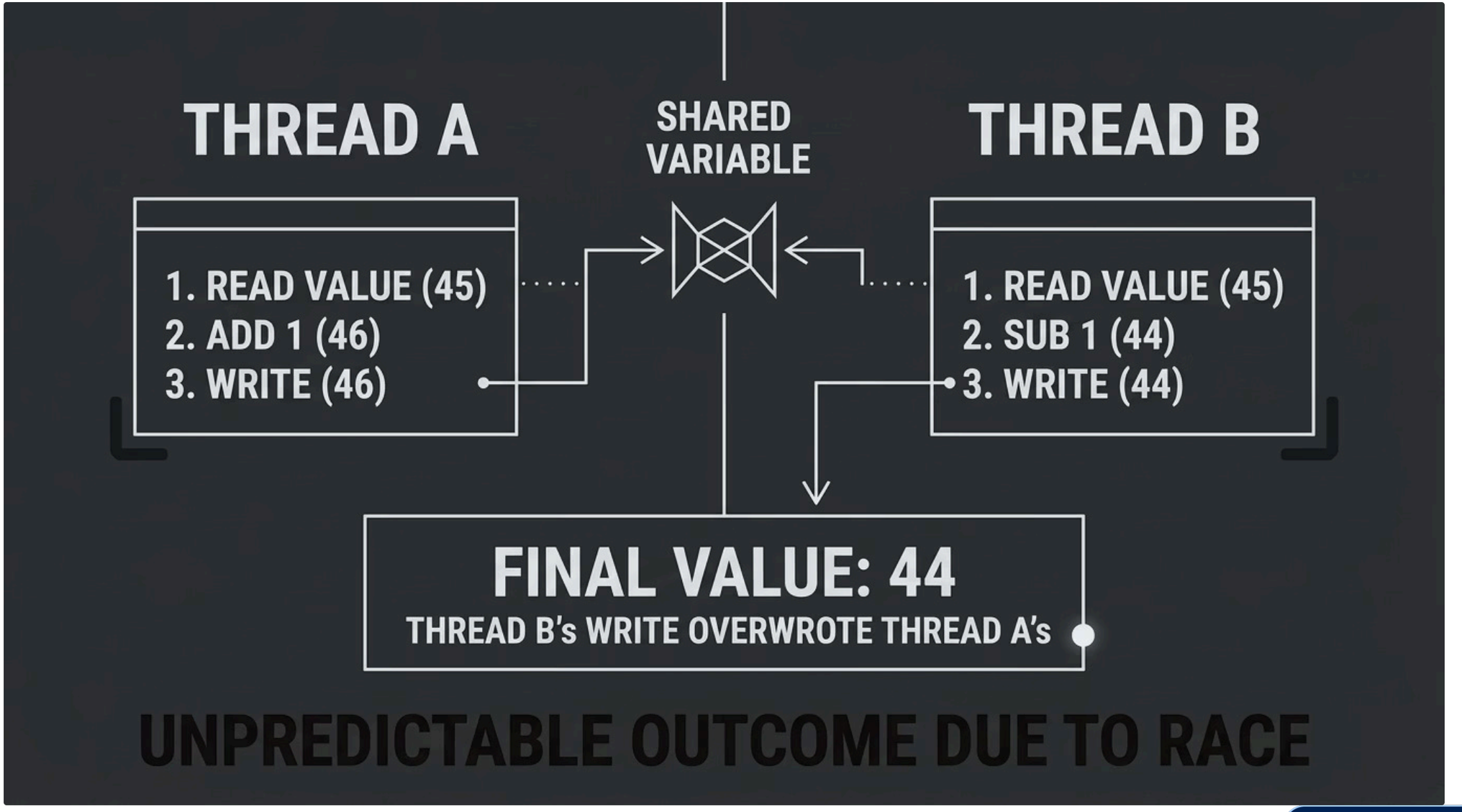
Critical Section Problem

Process synchronization techniques are crucial for maintaining the consistency of shared data. The critical section refers to code segments where processes or threads access shared resources and perform write operations. Due to concurrent execution, any process can be interrupted mid-execution, leading to potential data inconsistencies.



Major Thread Scheduling Issue: Race Condition

A race condition occurs when multiple threads access shared data and attempt to modify it simultaneously. The unpredictable nature of thread scheduling means the order of access is unknown, making the outcome dependent on the scheduling algorithm. Both threads "race" to access or change the data, leading to inconsistent results.



Made with GAMMA

Solutions to Race Conditions



Atomic Operations

Making critical code sections atomic ensures they execute in a single, uninterruptible CPU cycle, preventing race conditions.



Mutual Exclusion (Locks)

Locks enforce mutual exclusion, allowing only one thread or process to access a critical section at a time, thus avoiding race conditions.



Semaphores

Semaphores are signaling mechanisms used to control access to shared resources, preventing race conditions and ensuring proper synchronization.

While simple flag variables might seem like a solution, they are generally insufficient to solve the race condition problem effectively. Peterson's solution can address race conditions for two processes/threads, but it has limited scalability.

Made with GAMMA

Challenges with Mutexes/Locks

Contention

If one thread acquires a lock, other threads busy-wait. If the lock-holding thread dies, others can wait indefinitely.

Deadlocks

Locks can lead to deadlocks, where two or more threads are blocked indefinitely, waiting for each other to release resources.

Debugging Complexity

Debugging issues related to locks, such as deadlocks or race conditions, can be notoriously difficult and time-consuming.

Starvation

High-priority threads might suffer from starvation if lower-priority threads continuously acquire locks, preventing the high-priority threads from ever executing their critical sections.

Made with GAMMA